

STANDARD OPERATING PROCEDURE

Integrations, Webhooks and AI-Assisted Code

How we connect systems and write code with Claude
Code without shipping something we can't explain

EW-TECH-08 · v1.0 · 2026-09-10

Owner: Brad Palmer · For: Technical Builders, Web Developers,
Automation Devs



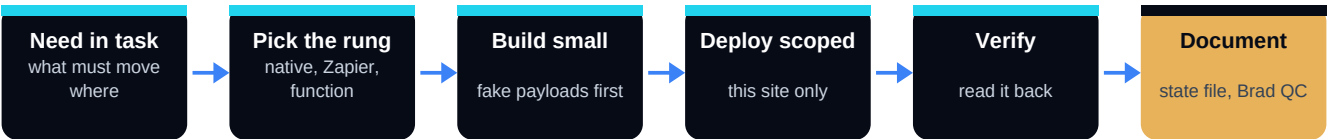
WHAT THIS IS

This is how we connect one system to another and how we write code with AI tools. You use it any time a task says webhook, function, script, API or integration, and any time you open Claude Code or a similar tool on Easyworks work. It produces a small, tested, documented piece of glue that the next person can read and explain.

DONE MEANS

The integration moves the right data the right way, a fake payload and a real one both behave, a retried webhook does not double write, every line can be explained by the person who shipped it, and the client state file says what it is and how to test it.

01 The whole flow



Every integration follows this path. The gold box is review. Only Brad or Cash mark it done.

Brad	Automation Dev	Technical Builder	Web Developer
Holds every token. Sets environment variables on sites. Approves scope and passes QC. Owns anything touching easyworks.ai or a client agent box.	Designs and builds functions and integrations. Owns the test payloads and the documentation.	Supplies the GHL side: workflows, webhook triggers, fields, tags. Confirms the contact looks right in GHL.	Wires site forms into the function pattern. See EW-TECH-07.

The best integration is the one you did not have to write. The second best is small enough to read in one screen. We climb the ladder from native to code and stop at the first rung that works.

Every piece of code we ship is read by a person who did not write it: Brad at QC, and whoever fixes it at two in the morning in six months. Write for them.

02 The decision ladder

Rung	Use it when	Do not use it when
1. Native GHL	GHL already does it: workflows, webhook triggers, calendars, forms, tags, the built in integrations for Facebook, Google, Stripe.	You are fighting the tool to make it do something it was not built for.
2. Zapier or Make	Two SaaS tools need a simple, linear hand off with no logic beyond mapping fields. The client is happy to pay for the seat.	There is branching, retries, dedupe, or anything you would need to explain with more than one sentence.
3. Serverless function	Anything with logic: validation, reshaping, idempotency, calling two systems, error handling, a secret that must stay server side.	Native or a Zap would do it in ten minutes.
4. A running service	Something must run on a schedule or stay connected. This is client agent territory, EW-TECH-10, and Brad decides.	Ever, on your own.

TIP

Write the rung you chose and why in the task before you build. If Brad would have picked a lower rung, that is a five minute conversation now and a rewrite later.

RULE

Zapier and Make accounts are the client's, in the client's name, paid by the client. We never run client automations on an Easyworks account.

03 GHL API v2 essentials for builders

Base URL	https://services.leadconnectorhq.com
Headers	Authorization: Bearer <token>, Version: 2021-07-28, Content-Type: application/json.
The token	A Private Integration Token (PIT) made in the sub account UI. One per client. Brad creates it and puts it in the site's environment variables. You read it from <code>process.env</code> , never from a file in the repo, never from chat.
Contacts	Upsert by phone: POST /contacts/upsert with <code>locationId</code> , <code>name</code> , <code>phone</code> , <code>email</code> , <code>tags</code> , <code>source</code> . Same phone twice, one contact.
Tags	POST /contacts/{id}/tags with { <code>tags</code> : [. . .] }. Tags are how workflows find the contact.
Notes	POST /contacts/{id}/notes with { <code>body</code> }. The write back pattern: say what happened on the record.
Opportunities	POST /opportunities/ with <code>pipelineId</code> , <code>locationId</code> , <code>pipelineStageId</code> , <code>contactId</code> , <code>status</code> . PUT /opportunities/{id} to move or value it.
Calendars	Free slots from the calendar's free slots endpoint. Block slots with a <code>calendarId</code> . The public booking widget URL is per calendar.
Money fields	A monetary custom field has <code>dataType</code> spelled MONETORY in the API. Yes, really.

What the API cannot do

- Create a pipeline. Pipelines are built in the UI or from a snapshot. Stage ids come from the UI after.
- Update the location profile with a sub account token. Reads work, updates return 401. That needs agency level access, which is Brad.
- Send email from a verified domain unless the domain is verified in Settings, Email Services. Otherwise it arrives from a relay address.

Rate limits and 429

GHl limits requests per location. When you get a 429, stop and back off. Retry at most three times with a growing wait (one second, two, four), honour a `Retry-After` header if present, and make sure a retry cannot double write. Log every 429 so we can see if a client is near the limit.

CONFIRM WITH BRAD

Exact rate limit numbers change with GHl releases and are not in this document on purpose. Design for 429 and you never need the number.

VERIFY BY READ BACK

Never trust a 2xx. After a write, read the record back and check the field actually changed. Some platforms return success and silently ignore the write. Verify by read back is the rule on every integration we ship.

04 The standard webhook function pattern

One shape for every function. Netlify Functions v2, one `.mjs` file, `export default async (req) => { ... }` returning a `Response`. Read it top to bottom and you know what it does.

1**Receive**

Accept only the method you expect. Anything else returns 405.
End state: a GET to a POST function returns 405, not a stack trace.

2**Validate**

Parse the JSON in a try. Require the fields you need (a phone or an email at minimum). Return 400 with a one line reason on bad input. If the sender signs webhooks, verify the signature before anything else.
End state: an empty body, a wrong type, and a missing field all return 400.

3**Check idempotency**

Every webhook can arrive twice. Take the event id from the payload (or hash the phone plus the event type plus a time bucket) and look it up in a store with strong consistency. If seen, return 200 and do nothing.
End state: sending the same payload twice creates one contact, one note, one opportunity.

4**Reshape**

Build the downstream body from the validated input. Name the fields after the target system. Keep the mapping in one small object so the next person can read it.
End state: the mapping fits on one screen.

5**POST downstream and await it**

Call the target API. Await it. Check `res.ok`. On failure, log the status and the body, and decide: retry (429, 5xx, up to three times with backoff) or give up and return 502 with a clear message.
End state: a fake downstream failure produces a logged error and a non 200 response, not a silent success.

6**Verify by read back**

Re read the record you wrote. Confirm the field, tag or stage is what you sent.
End state: the function only returns success if the read back matches.

7**Record the event id**

Write the event id to the store so a retry is a no op. Do this after the write succeeds, not before.

8**Respond**

Return a small JSON body with ok and the id you created. Never echo secrets, tokens or the full upstream error to the caller.
End state: the response is readable and safe to show in a log.

9**Log one line per event**

Timestamp, event id, what happened, status. Enough to reconstruct the story, never a full payload with personal data.

AWAIT EVERYTHING

Serverless tears the function down the moment you return. A write you did not await may never run and you will never see an error. Await every write, then respond. A little latency buys reliability.

RETRIES DOUBLE WRITE

Webhooks retry. Cron jobs overlap. Users double click. If your function is not idempotent, the client gets two texts, two contacts, two invoices. Idempotency is not optional on anything that writes.

Test with fake payloads before real ones

- Keep a `test/` folder with three files: `good.json`, `missing.json`, `garbage.json`. Post each with curl to the local dev server and to the preview.
- Post `good.json` twice. Expect one write.
- Point the downstream URL at a dead address once. Expect a logged error and a non 200.
- Only then send a real payload from GHSL or the site, and confirm the contact in the sub account with a screenshot.

05 AI assisted coding rules

We use Claude Code and similar tools on almost every task. They make good builders faster and bad builders dangerous. These rules are how we hire, how we review, and how we ship. Read them as one set.

YOU MUST BE ABLE TO EXPLAIN EVERY LINE

If you cannot explain a line in your own words to Brad on a call, it does not ship. Delete it or learn it. Copying what the tool produced is not building.

READ EVERY DIFF

The tool edits files you did not ask about. Read the whole diff before you commit, every time. Look for changed defaults, removed checks, and new dependencies you did not choose.

TEST BEFORE YOU TRUST

Run it. Feed it bad input. Watch the log. A function that looked fine is not a tested function. The AI will confidently invent an API shape, a method name, or a header. The test is how you catch it.

NAME WHAT THE AI GOT WRONG

Every Loom or task note on AI assisted work names at least one thing the tool got wrong and how you caught it. If you found nothing, you did not look. This is the question we hire on and the question Brad asks at QC.

NEVER PASTE SECRETS INTO AN AI TOOL

No token, password, key, customer list or client data goes into an AI tool, ever. Not in a prompt, not in a pasted log, not in a file the tool can read. If a secret touched a tool, tell Brad the same hour so it can be rotated.

ENV VARS ONLY

Code reads secrets from environment variables. `process.env.NAME`. Never a literal in the file, never a hard coded fallback that is a real value, never a `.env` file committed. Add the name to `.env.example` with an empty value so the next person knows it exists.

One more that is not a rule but a habit: ask the tool to explain its own code back to you before you read it, then check whether the explanation matches the code. When the two disagree you have found the bug.

06 Repo hygiene and deploy scopes

Every repo

- ☐ One repository per client under the easyworksai GitHub organisation. No client code in another client's repo, ever.
- ☐ A README that says what the site or function does, how to run it locally, the smoke test paths, and how to deploy.
- ☐ A `.env.example` listing every environment variable name with an empty value.
- ☐ `.env`, tokens, exports of client data and anything under `secrets` in `.gitignore` and never committed. If it was committed once, it is leaked, tell Brad.
- ☐ Small commits with plain messages saying what changed and why. One change per commit where you can.
- ☐ A branch for anything risky: a rewrite, a new integration, a change to a live form. Main stays deployable.
- ☐ No `node_modules`, no build output, no 2000px images in the repo.

Deploy scopes

What	Who can deploy	How
A client site on Netlify	The developer assigned, with per site deploy rights	From inside the project folder, preview alias first, production after Brad's QC, always with <code>--site</code> .
A client site's environment variables	Brad	You ask in the task. You never see the value.
easyworks.ai and Easyworks own functions	Brad only	Never a contractor, never a preview alias without Brad's say.
A client agent box	Brad only	See EW-TECH-10. Contractors do not deploy to the boxes.
Zapier or Make scenarios	The builder, on the client's account	Turn it on only after a test run with fake data is in the task.

TIP

A deploy you are not sure about is a deploy you do not run. Hit Blocked, name the site and the doubt. Brad would far rather answer a question than unpick a wrong deploy.

07 The documentation rule

Every integration gets one paragraph in the client's state file. Not a manual, a paragraph. Brad or GuapBot write the state file; you supply the paragraph in the task in this shape and it is copied in the same day.

What	One line on what it does. Site contact form to GHL contact with tag and opportunity.
Where	Repo name, file path, the Netlify site and function name, or the Zapier scenario name.
Secrets	The environment variable names it reads and where they are set. Names only, never values.
How to test	The exact curl or the exact click that proves it works, and what you expect to see in GHL.
Known limits	Anything it does not handle yet, and what happens if the downstream is down.

RULE

Written the same day the integration changes. An undocumented integration is a bug that has not happened yet.

08 Definition of done and when it goes wrong

Definition of done

- ☐ The rung and the reason are written in the task, and it is the lowest rung that works.
- ☐ Function follows the section 04 shape: receive, validate, idempotency, reshape, await the write, verify by read back, respond, log.
- ☐ Three fake payloads tested (good, missing field, garbage) plus a duplicate and a dead downstream. Results in the task.
- ☐ One real payload from the source system created the expected record in GHL, screenshot attached.
- ☐ No secret in the repo, in the task, in a Loom, or in an AI tool. `.env.example` updated.
- ☐ Deployed to the scoped place only, preview first, with `--site`.
- ☐ Loom or notes name what the AI got wrong and how you caught it.
- ☐ The one paragraph documentation is in the task, in the five part shape.
- ☐ Task moved to review. Brad or Cash marked it done.

When it goes wrong

Symptom	First three checks	Who to tell
Function returns 500	1. Netlify function logs for the stack trace. 2. Is the environment variable set on this site? 3. Does the payload match what validate expects?	Task notes. If a live form is down, Blocked and Brad now.
Contact created twice	1. Upsert, not create? 2. Idempotency store checked before the write and written after? 3. Is a GHL workflow also creating from the same trigger?	Task notes.
Write returned 200 but nothing changed	1. Read the record back. 2. Is the field name or stage id exactly right? 3. Are you in the right locationId?	Task notes, with the read back result.
429 from GHL	1. How many calls per event? 2. Is a loop paging with no cap? 3. Backoff and retry in place?	Task notes. If persistent, Brad, the client may be near the limit.
Downstream API is down	1. Did the function log it and return non 200? 2. Is the event id still unrecorded so a retry will work? 3. Can the source resend?	Task notes. Brad if a client is losing leads.
A secret was pasted somewhere it should not be	1. Stop. 2. Where exactly, and is it still there? 3. Nothing else, do not try to clean it up quietly.	Brad the same hour. It gets rotated. No blame for telling, only for hiding.
Netlify deploy did not pick up the function change	1. Deployed from the right folder with <code>--site</code> ? 2. Function bundle cached, use <code>--skip-functions-cache</code> . 3. Check the deploy log for the function name.	Task notes.